

Minimum Spanning Tree (MST)

What is a Minimum Spanning Tree?

In a [[Weighted Graph]] G , each edge e has an associated weight $w(e)$.

A spanning tree T is a subgraph of G that includes all vertices and is both connected and acyclic.

A Minimum Spanning Tree (MST) is a spanning tree whose total edge weight is as small as possible among all spanning trees of G .

![[weighted-graph.png]] > A *weighted graph* G

![[minimum-spanning-tree.png]] > The *minimum spanning tree associated with the weighted graph* G > (Total weight= $4 + 6 + 5 + 8 + 11 + 9 + 7 = 50$)

Applications of MSTs

Minimum spanning trees are widely used in network design, where the goal is to connect all points at minimum cost, such as in computer networks, road systems, telephone and electrical grids, circuits, cable TV infrastructure, and hydraulic systems. They are also important in approximation algorithms, for example as a building block in approaches to the Travelling Salesperson Problem and Steiner tree problems. Beyond these areas, MSTs appear in fields such as meteorology, cosmology, biomedical analysis, data encoding, and image analysis.

Properties of Minimum Spanning Trees

To simplify reasoning, we often assume that all edge weights are distinct and that the graph G is connected. Under these assumptions, an MST is guaranteed to exist and is unique. Even when weights are not distinct, the key structural properties below still hold and are the foundation of MST algorithms.

Cut Property

A cut is a partition of the vertex set into two non-empty groups. An edge is said to cross the cut if its endpoints lie in different groups. The cut property states that for any cut, the lightest edge crossing that cut must belong to the MST.

Intuition for the proof: assume the lightest cut edge e is not in the MST T . Since T connects all vertices, it must contain some other edge f that also crosses the same cut. Replacing f with e keeps the graph connected and acyclic (so it is still a spanning tree), but the total weight becomes smaller because $w(e) < w(f)$. This contradicts that T was minimum, so e must be in the MST.

![[cut-property.png]] ### Cycle Property

The cycle property states that for any cycle in the graph, the heaviest edge on that cycle cannot be part of the MST.

Intuition for the proof: suppose the heaviest edge f on some cycle is included in an MST T . If we remove f , the tree splits into two components. Because the original graph had a cycle, there is another edge e on that cycle that reconnects these components, and $w(e) < w(f)$ when weights are distinct. Replacing f with e gives a spanning tree of smaller total weight, again contradicting minimality. Therefore, the heaviest cycle edge is excluded from the MST.

![[cycle-property.png]]

Prim's Algorithm

Prim's algorithm builds an MST by growing a single tree from a chosen start vertex s . At each step, it adds the lightest edge that connects a vertex already in the tree to a vertex outside the tree. The process stops once the tree has $|V| - 1$ edges.

Prim's algorithm is a greedy algorithm: it makes the locally optimal choice at each step, and these choices lead to a globally optimal solution.

Why it works: at any step, the current tree defines a cut between vertices inside the tree and vertices outside it. The algorithm picks the lightest edge crossing that cut. By the cut property, this edge is safe to include in an MST. Repeating this argument at each step shows that after $|V| - 1$ additions, the resulting spanning tree is minimum.

![[prims-algorithm.png]]

Implementation Idea

Efficient implementations keep all not-yet-chosen vertices in a priority queue. For each vertex v , we store a key value: the minimum weight of any edge connecting v to the current tree. If no such edge is known yet, the key is set to ∞ .

In each iteration: - **EXTRACT-MIN** picks the vertex with smallest key and adds it to the tree. - For each adjacent vertex still outside the tree, we relax the connecting edge and perform **DECREASE-KEY** when we find a lighter connection.

Pseudocode

```
PRIM( $G, s$ )
  for each vertex  $v$  in  $V$ :
    key[ $v$ ] = infinity
    parent[ $v$ ] = null
  key[ $s$ ] = 0
```

```

P = priority queue containing all vertices in V, keyed by key[.]

while P is not empty:
    u = EXTRACT-MIN(P)
    for each edge (u, v) in Adj[u]:
        if v is in P and w(u, v) < key[v]:
            parent[v] = u
            key[v] = w(u, v)
            DECREASE-KEY(P, v, key[v])

```

Running Time

The operation counts are: - $|V|$ INSERT - $|V|$ EXTRACT-MIN - $O(|E|)$ DECREASE-KEY

So the total running time depends on the priority queue implementation:

Using $n = |V|$ and $m = |E|$:

Priority queue	INSERT	EXTRACT-MIN	DECREASE-KEY	Total
Array	$O(1)$	$O(n)$	$O(1)$	$O(n^2)$
Binary [[Heaps heap]]	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(m \log n)$
Fibonacci [[Heaps heap]]	$O(1)$ amortized	$O(\log n)$ amortized	$O(1)$ amortized	$O(m + n \log n)$

In practice, binary heaps are often preferred due to good constant factors and simpler implementation. # Kruskal's Algorithm Kruskal's algorithm builds an MST by repeatedly choosing the globally lightest edge that does not create a cycle with already chosen edges. Instead of growing one tree from a start vertex (as Prim does), Kruskal starts with a forest of single-vertex components and keeps merging components until only one spanning tree remains.

Kruskal's algorithm is also greedy: each step picks the cheapest safe edge available at that moment.

Why it works: when Kruskal considers an edge $e = (u, v)$ that connects two different components, that edge is the lightest edge crossing the cut between those components at that time. By the cut property, such an edge is safe to include in an MST. If an edge would connect vertices already in the same component, adding it would create a cycle, so it is skipped. After selecting $|V| - 1$ safe edges, the result is an MST.

![[kruskals-algorithm.png]]

Implementation Idea

The key operations are sorting edges by weight and efficiently checking whether adding an edge creates a cycle. This is typically done with a Union-Find (Disjoint Set Union, DSU) structure: - **MAKE-SET**(v): start each vertex in its own component. - **FIND**(v): return the representative of the component containing v . - **UNION**(u, v): merge two components.

For each edge in nondecreasing weight order, we add it only if its endpoints belong to different components.

Pseudocode

```
KRUSKAL(G)
  A = empty set of edges

  for each vertex v in V:
    MAKE-SET(v)

  sort edges E by nondecreasing weight

  for each edge (u, v) in sorted E:
    if FIND(u) != FIND(v):
      A = A union {(u, v)}
      UNION(u, v)
      if |A| = |V| - 1:
        break

  return A
```

Running Time

The total running time is dominated by: - Sorting edges: $O(|E| \log |E|)$ - Union-Find operations (**FIND**/**UNION**) over all edges: near-linear, typically $O(|E| \alpha(|V|))$ with path compression and union by rank

Overall:

$$O(|E| \log |E|) = O(|E| \log |V|)$$

for connected sparse graphs, since $|E| \leq |V|^2$ and logarithms are within constant factors.

In practice, Kruskal is often especially effective when edges are easy to sort and the graph is represented as an edge list.

Related

- [[Amortized Analysis]]
- [[Weighted Graph]]